Đánh giá Kiến trúc và Lộ trình Nâng cấp Ứng dụng Học Chữ Hán Theo Mô hình Hanly: Chinese Decoded

Mô hình tiếp cận giáo dục ngôn ngữ của ứng dụng *Hanly: Chinese Decoded* tạo nên chuẩn mực mới trong việc kết hợp giữa ngôn ngữ học tính toán chữ Hán (Sinological Computational Linguistics) và thiết kế giao diện tương tác lấy người dùng làm trung tâm. Việc nâng cấp một ứng dụng web học tiếng Trung thông thường lên mô hình giải mã Hán tự đa tầng, trực quan hóa từ nguyên và phân rã cấu trúc cấu kiện đòi hỏi sự tái thiết toàn diện từ cấu trúc lưu trữ dữ liệu, mô hình điều phối tiến độ ghi nhớ đến các kỹ thuật render đồ họa vector phía máy khách. Phân tích dưới đây làm rõ cơ chế vận hành của mô hình này, đánh giá tính khả thi kỹ thuật của kiến trúc thư mục đề xuất, khảo cứu hệ sinh thái dữ liệu mở và thiết lập chiến lược tối ưu hóa hiệu năng ứng dụng web lũy tiến (Progressive Web App - PWA).

Cơ chế Vận hành và Mô thức Thiết kế của Hanly: Chinese Decoded

Khác biệt cốt lõi giữa các ứng dụng thẻ ghi nhớ (flashcards) truyền thống và *Hanly: Chinese Decoded* nằm ở triết lý nhận thức học: Hanly không xem Hán tự là những khối hình học ngẫu nhiên cần ghi nhớ bằng phản xạ cơ học, mà xem mỗi chữ Hán là một hệ thống ký hiệu có tôn ti trật tự, được lắp ráp logic từ các thành phần hạt nhân.

Tiêu chí So sánh	Flashcard Truyền thống (Anki, Quizlet)	Mô hình Giải mã Ký tự của Hanly
Bản chất tri nhận	Ghi nhớ thụ động dựa trên hình thể chữ toàn vẹn	Kiến tạo chủ động từ các khối linh kiện độc lập
Thứ tự tiếp thu	Tuyến tính theo giáo trình bài học hoặc HSK ngẫu nhiên	Tối ưu hóa topo: nắm vững cấu kiện gốc trước khi mở chữ phức
Lớp giải nghĩa	Nghĩa tương đương ngữ nghĩa hiện đại	Tích hợp từ nguyên học, lịch sử tiến hóa và mnemomics
Tương tác giao diện	Thẻ tĩnh hai mặt (Mặt trước / Mặt sau)	Cây phân cấp động, tương tác chạm-mở đa tầng
Điều phối lặp lại	Hàng đợi ôn tập tách biệt khỏi không gian học	Trạng thái SRS nhúng trực tiếp vào toàn bộ giao diện duyệt

Triết lý học tập dựa trên cấu kiện (Component-based Learning)

Hanly vận dụng phương pháp phân rã cấu kiện lấy cảm hứng từ hệ thống James Heisig nhưng đã giải quyết được nhược điểm lớn nhất của phương pháp Heisig nguyên bản: sự cô lập khỏi tần suất sử dụng thực tế. Trong phương pháp Heisig truyền thống, người học buộc phải học nhiều chữ Hán cổ hoặc cực kỳ hiếm chỉ để phục vụ việc tạo thành phần biểu ý cho chữ khác. Hanly xây dựng một đồ thị có hướng (Directed Acyclic Graph) nhằm dung hòa giữa độ thường dùng trong HSK và tính phụ thuộc cấu tạo của chữ. Một chữ Hán phức chỉ được mở khóa khi

người học đã nắm vững tất cả các bộ thủ và cấu kiện thành phần của nó. Khi người dùng tương tác với một chữ như "妈" (mā), giao diện cho phép bóc tách trực tiếp thành bộ "女" (nữ - biểu thị nghĩa người phụ nữ) và thành phần "马" (mã - biểu thị âm đọc gợi ý), từ đó kích hoạt cơ chế truy nguyên cây phả hệ ký tự chỉ với một thao tác chạm.

Kiến trúc thẻ đa tầng và kỹ thuật kích thích đa giác quan

Thẻ hiển thị của Hanly không đơn thuần cung cấp mặt chữ và phiên âm, mà tích hợp bốn lớp thông tin đồng bộ. Tầng nhận diện trực quan đặt ký tự ở trọng tâm với kích thước lớn, đi kèm Pinyin có thanh điệu chuẩn hóa và phát âm khẩu ngữ bản xứ. Ngay bên dưới là tầng phân rã cấu trúc hiển thị cây quan hệ không gian, phân định rõ ràng vai trò của từng bộ phận: thành phần biểu âm (phonetic component) hay thành phần biểu ý (semantic radical).
Tầng diễn biến văn tự mô tả chuỗi tiến hóa hình thể từ Giáp cốt văn (Oracle Bone Script) thời nhà Thương, Kim văn (Bronze Script) thời Chu, Tiểu triện (Small Seal Script) thời Tần đến Khải thư hiện đại, giúp não bộ lý giải vì sao các nét chữ trừu tượng ngày nay lại bắt nguồn từ những hình vẽ mô phỏng tự nhiên. Cuối cùng, tầng liên tưởng hình ảnh cung cấp các bức họa nghệ thuật kỹ thuật số lồng ghép đường nét chữ Hán vào khung cảnh sinh động, kết hợp một câu chuyện mnemomic súc tích để khắc sâu ấn tượng vào vùng trí nhớ dài hạn.

Tích hợp công cụ Lặp lại Ngắt quãng (SRS) vào không gian giao diện

Thay vì giam giữ thuật toán lặp lại ngắt quãng trong một màn hình kiểm tra độc lập, Hanly nhúng trạng thái ghi nhớ của từng ký tự vào toàn bộ hệ thống định vị của ứng dụng. Dù người dùng đang tra cứu trong từ điển, duyệt cây cấu kiện hay đọc bài khóa, mọi chữ Hán đều phản ánh mức độ quen thuộc thông qua mã màu viền và huy hiệu trực quan. Cơ chế này mang lại khả năng tự đánh giá liên tục, cho phép người học nhận diện ngay lập tức những chữ sắp rơi vào vùng quên lãng trước khi bước vào các phiên ôn tập chuyên sâu.
Đánh giá Chi tiết Tính Khả thi của Cấu trúc Thư mục Đề xuất
Cấu trúc thư mục được đề xuất thể hiện tư duy phân tách tầng bài bản giữa mã nguồn logic, dữ liệu định dạng JSON và tài nguyên tĩnh. Dẫu vậy, dưới góc độ kỹ thuật ứng dụng web chuyên sâu, cấu trúc này tồn tại cả ưu điểm chiến lược lẫn các rủi ro vận hành cần tinh chỉnh.

Thành phần Cấu trúc	Đánh giá Chức năng	Phân tích Ưu điểm & Rủi ro Kỹ thuật
hanzi/js/	Module hóa bộ điều khiển	Tách biệt rành mạch các mối quan tâm (SoC); cần làm rõ cơ chế chia sẻ trạng thái
hanzi/data/characters/	Lưu trữ siêu dữ liệu chữ	Dễ bảo trì; chia nhỏ theo tệp HSK giúp giảm dung lượng tải ban đầu
hanzi/data/strokes/	Tệp nét viết đơn lẻ (u5988.json)	Tiết kiệm băng thông từng chữ nhưng tạo nút thắt cổ chai HTTP do phân mảnh yêu cầu
hanzi/assets/	Tài nguyên ảnh, vector, audio	Chuẩn hóa định danh Unicode hex tối ưu; cần chiến lược tải chậm (lazy-load)
lesson-adapter.js	Lớp tương thích ngược	Đảm bảo tính liên tục của hệ thống cũ (bai-16.js) mà không

Thành phần Cấu trúc	Đánh giá Chức năng	Phân tích Ưu điểm & Rủi ro Kỹ thuật
		làm ô nhiễm mã nguồn mới
sw.js & app.webmanifest	Nền tảng PWA ngoại tuyến	Cấu hình đầy đủ cho PWA; đòi hỏi chiến lược cache đa tầng để tránh quá tải bộ nhớ

Tính module hóa của các tệp JavaScript điều khiển

Sự phân rã các module JavaScript thể hiện tính độc lập cao. Module character.js tập trung thuần túy vào logic bóc tách dữ liệu cấu trúc và phả hệ, tạo điều kiện thuận lợi cho việc kiểm thử đơn vị độc lập với cây DOM. Module writer.js bao bọc các giao diện lập trình của thư viện vẽ nét, trong khi reader.js quản lý khả năng phân tích chuỗi văn bản và tra cứu từ ngữ cảnh. Module storage.js giữ vai trò của một lớp trừu tượng hóa dữ liệu (Data Access Layer), ngăn chặn sự phụ thuộc trực tiếp vào các giao diện lưu trữ trình duyệt cụ thể. Đặc biệt, sự xuất hiện của lesson-ad[span_60](start_span)[span_60](end_span)apter.js đóng vai trò là một mẫu thiết kế Adapter kinh điển, chuyển đổi các bài học tuyến tính cũ (data/hsk1/bai-16.js) thành các liên kết trỏ tới thực thể chữ Hán chi tiết mà không làm gián đoạn trải nghiệm của người dùng hiện tại.
Tuy nhiên, nếu các tệp JavaScript này được nạp tuần tự qua các thẻ <script> truyền thống trong môi trường trình duyệt, nguy cơ ô nhiễm không gian tên toàn cục (global scope pollution) và khó khăn trong việc chia sẻ trạng thái phản ứng (reactive state) giữa review.js, storage.js và app.js sẽ xuất hiện. Nhóm phát triển cần quy hoạch các tệp này dưới dạng ES Modules chuẩn (import/export) để đảm bảo tính đóng gói và khả năng tối ưu hóa tĩnh.

Rủi ro phân mảnh mạng từ cơ sở dữ liệu nét viết

Điểm nghẽn kỹ thuật lớn nhất nằm ở thiết kế thư mục strokes/ với các tệp định danh Unicode đơn lẻ như u5988.json. Mặc dù phương thức này giúp tải chính xác dữ liệu nét của chữ đang học, nó lại phát sinh hiện tượng bùng nổ số lượng yêu cầu mạng vi mô (micro-HTTP requests). Khi người học bước vào phiên ôn tập SRS lướt qua từ 30 đến 50 chữ liên tục, trình duyệt sẽ phải khởi tạo hàng chục kết nối TCP/TLS đồng thời. Trên các thiết bị di động sử dụng mạng viễn thông có độ trễ cao, hiện tượng nghẽn hàng đợi (queue blocking) sẽ gây ra độ trễ hiển thị nét vẽ rõ rệt, phá vỡ tính mượt mà của giao diện.

Quản trị tài nguyên tĩnh và chiến lược PWA ngoại tuyến

Việc quy ước tên tệp minh họa (u5988.webp) và tiến hóa văn tự (u76ee.svg) theo mã điểm Unicode (Code Point Hexadecimal) là phương pháp chuẩn hóa trong kỹ thuật phần mềm CJK. Cơ chế này triệt tiêu hoàn toàn rủi ro lỗi mã hóa ký tự (encoding mismatches) trên các hệ thống tệp khác nhau, đồng thời cho phép tạo đường dẫn động trực tiếp từ hàm biến đổi ký tự trong JavaScript ('妈'.charCodeAt(0).toString(16)) mà không cần bảng ánh xạ phụ trợ.
Đối với tệp điều phối bộ nhớ đệm sw.js, khối lượng tài nguyên bao gồm âm thanh, hình ảnh minh họa độ phân giải cao và đồ họa vector đòi hỏi một chiến lược phân chia bộ nhớ nghiêm ngặt. Nếu Service Worker cố gắng lưu trữ toàn bộ các tệp đa phương tiện vào bộ nhớ đệm ngay khi cài đặt (pre-caching), quá trình cài đặt PWA sẽ thất bại hoặc gây lãng phí nghiêm trọng dung lượng lưu trữ của thiết bị. Cần thiết lập chiến lược *Cache First* có giới hạn dung lượng tối đa và thời gian hết hạn cho các tệp WebP và Audio, trong khi áp dụng *Stale While Revalidate*

cho các tệp siêu dữ liệu JSON.
Khảo sát Hệ sinh thái Dữ liệu Mở và Tương thích Lược đồ
Thay vì xây dựng lại từ đầu hệ thống dữ liệu đồ sộ của hàng nghìn Hán tự, việc kế thừa các kho dữ liệu mở có nguồn gốc học thuật chuẩn xác là giải pháp tối ưu hóa nguồn lực phát triển.

Nguồn Dữ liệu / Thư viện	Định dạng Dữ liệu Gốc	Ứng dụng Mục tiêu trong Dự án	Điều kiện Bản quyền
Make Me a Hanzi	dictionary.txt, graphics.txt	Cung cấp tọa độ SVG nét viết, thứ tự nét, etymology, IDS	Giấy phép Kép: LGPL-3.0 & Arphic Public License
Hanzi Writer	JavaScript Library & Data Engine	Kết xuất hoạt ảnh thứ tự nét, chấm điểm bài tập viết tương tác	Giấy phép MIT (mã nguồn) & Arphic (dữ liệu nét)
CJKVI-IDS Database	Plain text / Tab-separated	Chuỗi phân rã cấu trúc chữ Hán đệ quy (Ideographic Sequences)	Public Domain / Giấy phép Mở CHISE
Ancient Chinese Characters Project	SVG vector (Wikimedia Commons)	Đồ họa Giáp cốt văn, Kim văn, Tiểu triện	Public Domain / Creative Commons CC0
ts-fsrs	Thư viện TypeScript / JavaScript	Động cơ điều phối thuật toán lặp lại ngắt quãng hiện đại FSRS	Giấy phép MIT

Cơ chế tích hợp Hanly-like với Make Me a Hanzi và Hanzi Writer

Kho dữ liệu *Make Me a Hanzi* là xương sống cho hầu hết các ứng dụng xử lý chữ Hán hiện đại. Dữ liệu được chia thành hai tệp chính nhằm phân định ranh giới bản quyền: dictionary.txt lưu trữ các thông tin ngôn ngữ như định nghĩa, pinyin, bộ thủ, chuỗi phân giải cấu trúc (IDS) và từ nguyên; trong khi graphics.txt cung cấp danh sách các chuỗi SVG biểu diễn từng nét vẽ độc lập cùng tọa độ đường trung vị (stroke medians).
Tất cả các nét vẽ được số hóa trên hệ tọa độ chuẩn 1024 \times 1024, trong đó điểm gốc tọa độ (0, 900) nằm ở góc trên bên trái và trục tung đảo ngược theo chiều dương hướng lên trên. Thư viện *Hanzi Writer* đóng gói sẵn bộ chuyển đổi ma trận để tái dựng tọa độ này trên canvas HTML5 hoặc SVG DOM. Nhóm phát triển có thể thiết lập hàm nạp dữ liệu cục bộ thông qua tùy chọn charDat[span_51](start_span)[span_51](end_span)aLoader, cho phép ứng dụng đọc dữ liệu trực tiếp từ hệ thống tệp nội bộ mà không cần phụ thuộc vào mạng phân phối nội dung bên ngoài.

Phân tích Cấu trúc Chữ Hán bằng Chuỗi Ký hiệu IDS

Để tái tạo trải nghiệm khám phá các khối cấu kiện như trò chơi lắp ráp Lego của Hanly, module character.js cần đọc và xử lý chuỗi mô tả ký tự tượng hình (Ideographic Description Sequences - IDS) từ chuẩn Unicode (U+2FF0 đến U+2FFB). Chuỗi IDS mô tả cấu trúc không gian của chữ thông qua các toán tử tiền tố:

- ⿰ (U+2FF0): Bố cục song song hai thành phần trái - phải, ví dụ ⿰女马 hình thành nên "妈".
- ⿱ (U+2FF1): Bố cục hai thành phần trên - dưới, ví dụ ⿱艹化 hình thành nên "花".
- ⿴ (U+2FF4): Bố cục bao bọc toàn phần, ví dụ ⿴口玉 hình thành nên "国".

- ⿴ (U+2FF5): Bố cục bao bọc ba phía từ trên xuống, ví dụ ⿴門日 hình thành nên "間".

Bằng cách xây dựng một bộ phân tích cú pháp đệ quy đơn giản cho chuỗi IDS, ứng dụng có thể tự động bóc tách một chữ Hán phức tạp thành một cây cú pháp trừu tượng, từ đó kết xuất thành các nút bấm linh kiện tương tác trên giao diện, cho phép người dùng đào sâu vào từng thành phần cấu tạo.

Ứng dụng Thuật toán FSRS Thay thế SM-2

Thay vì triển khai thuật toán SM-2 cổ điển vốn dựa trên các hệ số cố định thiếu linh hoạt và dễ dẫn đến tình trạng dồn ứ thẻ ôn tập (Ease Hell), việc tích hợp thuật toán *Free Spaced Repetition Scheduler* (FSRS) thông qua thư viện ts-fsrs mang lại hiệu quả vượt trội trong việc kiểm soát tiến trình ghi nhớ. FSRS xây dựng trên mô hình trí nhớ ba thành phần DSR: Độ khó (Difficulty - D), Độ bền vững (Stability - S), và Khả năng truy hồi (Retrievability - R). Xác suất người học nhớ lại thành công một ký tự sau khoảng thời gian t ngày được tính toán qua hàm suy giảm lũy thừa âm:
$$R(t, S) = \left(1 + \frac{t}{9 \cdot S}\right)^{-1}$$
Hàm số này mô hình hóa chính xác hiện tượng suy giảm trí nhớ thực tế. Khi người học hoàn thành bài ôn tập, thuật toán tự động cập nhật độ bền vững S và độ khó D dựa trên mức đánh giá (Again, Hard, Good, Easy), từ đó xác định chính xác ngày ôn tập tiếp theo để duy trì mức độ lưu giữ mục tiêu (thường mặc định ở mức 90\%). Việc áp dụng FSRS giúp hệ thống phân phối lịch học thông minh hơn, triệt tiêu gánh nặng ôn tập thừa thãi cho những ký tự dễ nhớ.

Tối ưu hóa Lưu trữ Phía Trình duyệt và Hiệu năng Tải trang

Để đảm bảo ứng dụng vận hành với tốc độ khung hình ổn định (60\text{ fps}) trên các thiết bị di động tầm trung, toàn bộ kiến trúc lưu trữ và xử lý đa phương tiện cần được chuyển dịch sang mô hình ngoại tuyến ưu tiên (offline-first architecture).

Loại Dữ liệu	Khối lượng Ước tính	Giải pháp Lưu trữ Khuyến nghị	Cơ chế Quản lý Vòng đời & Tải trang
Dữ liệu Nét viết (Strokes)	\sim 2.5\text{ KB} / chữ	IndexedDB (Object Store: strokes)	Đóng gói theo cấp độ (Bundle-and-Seed); truy xuất cục bộ không trễ
Cấu trúc, Nghĩa & Từ nguyên	\sim 450\text{ KB} (Toàn bộ HSK 1-3)	IndexedDB (Object Store: characters)	Nạp sẵn khi người dùng cài đặt ứng dụng qua Service Worker
Dữ liệu Lịch sử & SRS	\sim 200\text{ bytes} / bản ghi	IndexedDB (Object Store: srs_state)	Cập nhật nguyên tử (Atomic transaction); xuất tệp sao lưu JSON
Tranh Minh họa Gợi nhớ	20 - 35\text{ KB} / hình	Service Worker Cache API	Lazy-load qua IntersectionObserver khi thẻ tiến vào viewport
Vector Tiến hóa Văn tự	1[span_63](start_span)[span_63](end_span).5	Service Worker Cache API	Cache tĩnh song hành cùng bài học chứa ký

Loại Dữ liệu	Khối lượng Ước tính	Giải pháp Lưu trữ Khuyến nghị	Cơ chế Quản lý Vòng đời & Tải trang
	- 3\text{ KB} / tệp SVG		tự tương ứng
Tệp Phát âm Khẩu ngữ	10 - 20\text{ KB} / từ	Web Audio API Buffer Memory	Nạp theo nhu cầu phát (on-demand); cache vào bộ nhớ giải mã

Chiến lược Đóng gói và Gieo Dữ liệu Cục bộ (Bundle-and-Seed)

Để loại bỏ triệt để vấn đề phân mảnh mạng khi duy trì hàng nghìn tệp JSON nét viết riêng lẻ trong thư mục strokes/, giải pháp tối ưu là chuyển sang cơ chế *Bundle-and-Seed*. Dữ liệu nét viết của từng cấp độ HSK sẽ được nén gọn thành một tệp gói duy nhất, ví dụ hsk1-strokes.json với dung lượng chỉ khoảng 300\text{ KB} khi nén gzip cho toàn bộ cấp độ.
Khi ứng dụng được khởi chạy lần đầu tiên, module storage.js sẽ kích hoạt tiến trình kiểm tra cơ sở dữ liệu IndexedDB. Nếu bảng dữ liệu chưa tồn tại, một web worker ngầm sẽ tải tệp gói này về, bóc tách từng ký tự và lưu vào Object Store strokes với khóa chính là ký tự hoặc mã Unicode hex (u5988). Sau thời điểm này, mọi truy vấn nét viết từ writer.js đều diễn ra hoàn toàn trên bộ nhớ IndexedDB với thời gian đáp ứng dưới 5\text{ ms}, giải phóng ứng dụng khỏi mọi phụ thuộc vào kết nối mạng.

Tối ưu hóa Luồng Âm thanh và Hiển thị Đồ họa

Việc khởi tạo liên tục các phần tử <audio> trong DOM mỗi khi phát âm thanh từ vựng thường dẫn đến rò rỉ bộ nhớ và độ trễ phản hồi trên trình duyệt di động. Module âm thanh cần được hiện thực hóa thông qua giao diện AudioContext của Web Audio API. Một bộ đệm ánh xạ (Map<string, AudioBuffer>) sẽ giữ lại dữ liệu nhị phân đã giải mã của các từ vựng đang học, cho phép phát âm thanh tức thì mà không gây giật lag giao diện.
Đối với hình ảnh minh họa định dạng WebP, việc tải cần được trì hoãn thông qua API IntersectionObserver. Hình ảnh chỉ được tải về và giải mã khi khung thẻ Hán tự thực sự trượt vào vùng hiển thị của người dùng, giúp tiết kiệm đáng kể băng thông mạng và năng lượng xử lý đồ họa của thiết bị di động.

Kiến trúc Thư mục Tinh chỉnh và Đặc tả Lược đồ Dữ liệu

Dựa trên các phân tích kỹ thuật và yêu cầu vận hành, cấu trúc thư mục được tinh chỉnh lại nhằm bảo đảm khả năng mở rộng quy mô dữ liệu, chuẩn hóa phân tầng module và tương thích hoàn toàn với các chuẩn web hiện đại.

Hệ thống thư mục tinh chỉnh

- index.html: Cổng điều hướng chính của nền tảng học tập
- manifest.js: Chỉ mục danh mục khóa học và bài học hiện hữu
- data/
    - hsk1/
        - bai-16.js: Tệp dữ liệu bài học cũ được duy trì nguyên vẹn
- hanzi/

- index.html: Không gian học tập và giải mã Hán tự
- app.webmanifest: Tệp cấu hình PWA Manifest
- sw.js: Service Worker điều phối chiến lược bộ nhớ đệm
- css/
  - base.css: Định nghĩa biến màu sắc, typography và chuẩn hóa giao diện
  - layout.css: Hệ thống khung lưới, thanh điều hướng và vùng làm việc
  - cards.css: Khung hiển thị thẻ chữ, hiệu ứng lật và animation
  - writer.css: Bố cục ô lưới tập viết (Điền tự cách / Mễ tự cách)
  - mobile.css: Tối ưu hóa vùng an toàn (safe-area) và cử chỉ cảm ứng trên thiết bị di động
- lib/
  - hanzi-writer.min.js: Thư viện xử lý nét viết SVG
  - fsrs.min.js: Động cơ thuật toán lặp lại ngắt quãng FSRS
  - idb-keyval.min.js: Tiện ích tối giản thao tác với IndexedDB
- js/
  - app.js: Điểm khởi nhập (Entrypoint), quản lý trạng thái giao diện và định tuyến
  - characte[span_137](start_span)[span_137](end_span)r.js: Xử lý phân tích chuỗi IDS và dựng cây cấu kiện
  - writer.js: Quản lý hoạt ảnh và chấm điểm luyện viết tương tác
  - reader.js: Module đọc văn bản và tra từ tại chỗ
  - review.js: Bộ điều phối lịch ôn tập theo thuật toán FSRS
  - storage.js: Quản lý kết nối IndexedDB và đồng bộ dữ liệu ngoại tuyến
  - lesson-adapter.js: Cầu nối chuyển đổi dữ liệu bài học cũ sang thẻ Hán tự mới
- data/
  - bundles/
    - hsk1-core.json: Tập hợp thông tin ngữ nghĩa, cấu trúc và từ nguyên HSK1
    - hsk1-strokes.json: Tập hợp tọa độ nét viết đóng gói cho toàn bộ cấp HSK1
  - components.json: Từ điển bộ thủ và cấu kiện đồ họa dùng chung
  - evolution.json: Danh mục ánh xạ hình ảnh tiến hóa của các ký tự * assets/
  - illustrations/: Tranh minh họa gợi nhớ WebP theo mã Unicode (u5988.webp)
  - evolution/: Vector SVG diễn biến văn tự (u5988-oracle.svg, u5988-seal.svg)
  - audio/: Tệp âm thanh khẩu ngữ định dạng MP3/Opus chuẩn hóa
  - icons/: Bộ biểu tượng ứng dụng phục vụ PWA
  - fonts/: Phông chữ Hán chuẩn Khải thư (Kaiti) phục vụ hiển thị chính xác
- LICENSES.md: Văn bản ghi nhận tác quyền và điều khoản các kho dữ liệu nguồn mở

Đặc tả lược đồ JSON chuẩn hóa cho thực thể chữ Hán

Để đáp ứng đồng thời yêu cầu bóc tách cấu kiện kiểu Lego, trực quan hóa diễn biến từ nguyên, hướng dẫn nét viết và điều phối SRS, mỗi ký tự trong tệp gói hsk1-core.json được định dạng theo cấu trúc dữ liệu chặt chẽ:

```
{
  "character": "妈",
  "unicode": "5988",
  "pinyin": ["mā"],
```

```
  "meaning_vi": "mẹ, má",
  "hsk_level": 1,
  "frequency_rank": 452,
  "stroke_count": 6,
  "radical": "女",
  "structure": {
    "ids": "⿰女马",
    "type": "pictophonetic",
    "description": "Chữ hình thanh kết hợp giữa ý nghĩa và âm thanh",
    "tree": {
      "operator": "⿰",
      "components": [
        {
          "character": "女",
          "role": "semantic",
          "hint_vi": "Người phụ nữ (thành phần mang ý nghĩa người mẹ)"
        },
        {
          "character": "马",
          "role": "phonetic",
          "hint_vi": "Ngựa (mǎ đóng vai trò gợi ý âm đọc cho mā)"
        }
      ]
    }
  },
  "etymology": {
    "evolution_available": ["oracle", "bronze", "seal"],
    "story_vi": "Người phụ nữ (女) luôn làm việc vất vả, bền bỉ như
chú ngựa (马) để chăm sóc con cái chính là Mẹ (妈)."
  },
  "compound_words": [
    {
      "word": "妈妈",
      "pinyin": "māma",
      "meaning_vi": "mẹ, má"
    }
  ],
  "assets": {
    "illustration": "illustrations/u5988.webp",
    "audio": "audio/ma1.mp3"
  }
}
```

Lược đồ này đồng bộ hoàn toàn với dữ liệu phân rã của Make Me a Hanzi và CJKVI-IDS, cho phép module character.js tự động dựng giao diện phân tầng mà không cần xử lý chuỗi thủ công.

Lộ trình Triển khai Kỹ thuật Từng bước

Việc hiện thực hóa dự án cần được phân kỳ thành năm giai đoạn kế tiếp nhau, bảo đảm hệ thống bài học cũ tiếp tục hoạt động ổn định trong suốt quá trình phát triển tính năng mới.

Giai đoạn 1: Chuẩn hóa Dữ liệu và Xây dựng Đường ống ETL

Công việc khởi đầu tập trung vào việc xử lý nguồn dữ liệu thô từ kho lưu trữ Make Me a Hanzi. Nhóm phát triển thiết lập một kịch bản dòng lệnh tự động (Node.js hoặc Python) để bóc tách tệp dictionary.txt và graphics.txt, trích xuất toàn bộ các ký tự thuộc danh mục HSK 1. Quá trình này sẽ làm sạch các chuỗi IDS, trích xuất cấu kiện thành phần và ghép nối tọa độ nét viết tương ứng.
Song song với đó, các tệp vector diễn biến ký tự (Giáp cốt văn, Tiểu triện) từ dự án Wikimedia Commons Ancient Chinese characters project sẽ được thu thập và đổi tên theo chuẩn Unicode hex. Kết quả đầu ra của giai đoạn này là hai tệp gói hoàn chỉnh: hsk1-core.json và hsk1-strokes.json.

Giai đoạn 2: Xây dựng Giao diện Tương tác và Giải mã Cấu trúc

Giai đoạn tiếp theo tập trung vào việc phát triển giao diện người dùng và thuật toán phân rã cấu trúc. Module character.js được hiện thực hóa với bộ phân tích cú pháp chuỗi IDS, có nhiệm vụ chuyển đổi cấu trúc không gian thành các phần tử HTML tương tác. Giao diện thẻ đa tầng trong cards.css được định hình theo ngôn ngữ thiết kế tối giản, tập trung vào độ tương phản cao và hiệu ứng chuyển cảnh mượt mà tương tự Hanly.
Thanh tiến trình diễn biến chữ Hán (Evolution Bar) được tích hợp ở vị trí nổi bật, cho phép người dùng quan sát trực quan sự biến đổi hình thái của ký tự từ thời cổ đại đến dạng chuẩn hóa hiện nay.

Giai đoạn 3: Tích hợp Động cơ Viết nét và Hệ thống Đa phương tiện

Khi khung giao diện đã hoàn thiện, module writer.js sẽ kết nối với thư viện Hanzi Writer. Hệ thống thiết lập hai chế độ hoạt động chính: chế độ trình diễn (Animation Mode) cho phép người học quan sát thứ tự và hướng đi của từng nét vẽ; chế độ tập viết (Quiz Mode) cung cấp bề mặt tương tác trên ô lưới Điền tự cách, tự động phát hiện lỗi sai thứ tự nét và cung cấp gợi ý thông minh sau ba lần vẽ lỗi.
Cùng thời điểm này, module âm thanh ứng dụng Web Audio API được kích hoạt để phát âm thanh Pinyin chuẩn xác, và các hình ảnh tranh vẽ mnemomic định dạng WebP được nhúng vào phần chân thẻ dưới cơ chế nạp chậm.

Giai đoạn 4: Cài đặt Hệ thống Lặp lại Ngắt quãng và Lưu trữ Nội bộ

Giai đoạn này hiện thực hóa lớp lưu trữ dữ liệu bền vững và động cơ ghi nhớ. Module storage.js khởi tạo cơ sở dữ liệu IndexedDB trên trình duyệt, nạp tự động dữ liệu từ các tệp gói vào các bảng lưu trữ cục bộ. Module review.js tích hợp thuật toán FSRS thông qua thư viện ts-fsrs, quản lý việc tính toán khoảng cách ngày ôn tập dựa trên độ bền vững (S) và độ khó (D). Trạng thái ghi nhớ của người học được đồng bộ ngược lại vào giao diện danh mục, gán các

viền chỉ báo tiến độ lên từng chữ Hán để tạo động lực học tập liên tục.

Giai đoạn 5: Tương thích Hóa Hệ thống Cũ, PWA và Hoàn thiện Pháp lý

Giai đoạn cuối cùng bảo đảm tính liên kết hệ thống và sự sẵn sàng phát hành. Module lesson-adapter.js được viết để đọc tệp bài học cũ data/hsk1/bai-16.js. Bộ chuyển đổi này tự động quét các từ vựng trong bài 16, tạo các liên kết sâu (deep links) dẫn thẳng vào chế độ giải mã chữ Hán tương ứng trong không gian hanzi/, cho phép người học vừa học bài khóa vừa tra cứu chiết tự chuyên sâu.
Tệp Service Worker (sw.js) và app.webmanifest được tinh chỉnh cấu hình hoàn chỉnh nhằm bảo đảm khả năng cài đặt ứng dụng lên màn hình chính thiết bị di động và duy trì hoạt động ngoại tuyến ổn định. Cuối cùng, tệp LICENSES.md được lập để ghi nhận đầy đủ bản quyền của Make Me a Hanzi (LGPL-3.0/Arphic), Hanzi Writer (MIT), và Wikimedia Commons theo đúng quy chuẩn pháp lý quốc tế.

Khuyến nghị Kiến trúc Tổng hợp

Mô hình chuyển đổi ứng dụng học tiếng Trung sang hình thái giải mã ký tự trực quan như *Hanly: Chinese Decoded* tạo ra bước nhảy vọt về chất lượng tiếp thu cho người học nhờ việc gắn kết chặt chẽ giữa ngôn ngữ học từ nguyên và khoa học nhận thức. Ba khuyến nghị mang tính nguyên tắc nhóm phát triển cần tuân thủ nghiêm ngặt trong quá trình thi công gồm có:
Thứ nhất, kiên quyết loại bỏ mô hình phân tán hàng nghìn tệp JSON nét viết riêng rẽ trên máy chủ HTTP để chuyển hoàn toàn sang cơ chế đóng gói theo cấp độ (Bundle-and-Seed) kết hợp lưu trữ chỉ mục IndexedDB, triệt tiêu mọi nguy cơ nghẽn kết nối mạng và đảm bảo độ phản hồi giao diện tức thì.
Thứ hai, chuẩn hóa dữ liệu chiết tự dựa trên các thông số ngôn ngữ học mở đã được quốc tế công nhận (Make Me a Hanzi và CJKVI-IDS), tránh việc tự định nghĩa cấu trúc phân rã thủ công để đảm bảo tính chính xác và khả năng mở rộng dữ liệu lên hàng nghìn Hán tự nâng cao trong tương lai.
Thứ ba, thay thế các thuật toán lặp lại ngắt quãng đơn giản bằng động cơ FSRS hiện đại, đưa chỉ số trạng thái ghi nhớ vào mọi ngóc ngách của giao diện người dùng, giúp chuyển hóa toàn bộ ứng dụng thành một mạng lưới học tập tương tác, trực quan và liền mạch.

Nguồn trích dẫn

1. Hanly: Chinese Decoded - Apps on Google Play, https://play.google.com/store/apps/details?id=app.hanly.chinese 2. Hanly: Chinese Decoded for Android - Free App Download - AppBrain, https://www.appbrain.com/app/hanly-chinese-decoded/app.hanly.chinese 3. I hated how other apps teach the characters, so I built my own. - Reddit, https://www.reddit.com/r/ChineseLanguage/comments/1j1tzry/i_hated_how_other_apps_teach_the_characters_so_i/ 4. Best Practices for Persisting Application State with IndexedDB, https://web.dev/articles/indexeddb-best-practices-app-state 5. skishore/makemeahanzi: Free, open-source Chinese character data, https://github.com/skishore/makemeahanzi 6. Commons:Ancient Chinese characters project, https://commons.wikimedia.org/wiki/Commons:Ancient_Chinese_characters_project 7.

Optimizing JSON, IndexedDB, or postMessage - Andrea Giammarchi, https://webreflection.medium.com/optimizing-json-indexeddb-or-postmessage-698112860899 8. Generative Sawndip | stevenzhou01 - Observable Notebooks, https://observablehq.com/@stevenzhou01/cultural-identity-and-new-media-project 9. What are some best practices for working with IndexedDB in, https://www.mindstick.com/interview/34318/what-are-some-best-practices-for-working-with-indexeddb-in-production 10. A reliable modern dictionary for recursively decomposing characters, https://www.reddit.com/r/ChineseLanguage/comments/1eor0ue/a_reliable_modern_dictionary_for_recursively/ 11. Related projects - KanjiVG, https://kanjivg.tagaini.net/related.html 12. Docs - Hanzi Writer, https://hanziwriter.org/docs.html 13. The Algorithm · open-spaced-repetition/awesome-fsrs Wiki - GitHub, https://github.com/open-spaced-repetition/awesome-fsrs/wiki/The-Algorithm 14. ABC of FSRS · open-spaced-repetition/awesome-fsrs Wiki - GitHub, https://github.com/open-spaced-repetition/awesome-fsrs/wiki/ABC-of-FSRS 15. Awesome FSRS - open-spaced-repetition - GitHub, https://github.com/open-spaced-repetition/awesome-fsrs 16. @squeakyrobot/fsrs - npm, https://www.npmjs.com/package/@squeakyrobot/fsrs 17. GitHub - open-spaced-repetition/ts-fsrs: ts-fsrs is a versatile package, https://github.com/open-spaced-repetition/ts-fsrs 18. File:馬-seal.svg - Wikimedia Commons, https://commons.wikimedia.org/wiki/File:%E9%A6%AC-seal.svg 19. Chinese Stroke Order Generator (GIF) - GitHub, https://github.com/Yark-yao/Chinese-Stroke-Order-Generator 20. Open Source Licenses | DracoChinese, https://www.dracochinese.com/licenses 21. License - Hanzi Guide, https://www.hanzi.guide/license.html 22. hanzi-writer-data/stroke_data_parser.py at master · chanind/hanzi, https://github.com/chanind/hanzi-writer-data/blob/master/stroke_data_parser.py 23. Ancient Glyph Attribution — Wikimedia Commons Credits | HanziWise, https://www.hanziwise.com/glyph-attribution/